

Module Explanation

build_s4g_geometry_manifest.py

This document explains the Python module that builds a CSV geometry manifest for downloaded S4G galaxy FITS images. It follows the code structure and summarizes what each major function contributes to the data pipeline.

Big Picture

- Read the local scrambled galaxy map.
- Read the local s4gbars_table.dat file.
- Optionally fetch and cache public VizieR catalogues.
- For each galaxy, locate the FITS file, read image metadata, merge catalogue rows by name, and assemble one normalized output row.
- Write the final manifest to geometry_output/s4g_image_geometry_manifest.csv.

Configuration

The constants near the top of the module define default paths, catalogue identifiers, missing-value rules, and the final CSV schema.

Default paths

The script points to the local scrambled map, local S4G table, downloaded 3.6 micron image directory, output directory, and catalogue cache directory.

Catalogues

The VizieR IDs refer to Herrera-Endoqui bar data, Salo galaxy data, and Diaz-Garcia Fourier measurements.

OUTPUT_FIELDS

This list is the exact output CSV schema. Every generated row is initialized with these fields before values are filled in.

Data Cleaning

`clean_value()` converts text values into numbers where possible and normalizes placeholder values into `None`. Values such as -99, -999, and -999.999 are treated as missing. Some size-like columns also treat zero or negative values as missing because those would not be meaningful physical sizes.

```
def clean_value(value, column=None):  
    # numeric missing codes -> None
```

```
# zero or non-positive size values -> None
# otherwise return a number or original string
```

Local File Readers

read_scrambled_map()

Reads scrambled_map.txt and returns rows with i_scrambled, i_orig, and galaxy name. Blank lines and comments are skipped.

read_s4g_table()

Finds the commented header beginning with name, then parses subsequent data rows into dictionaries keyed by galaxy name. Each value passes through clean_value().

VizieR Catalogue Handling

The public catalogue path is cache-aware. If a cached ECSV table exists and cache use is enabled, the script reads from disk. Otherwise it fetches the catalogue through astroquery.vizier, writes an ECSV cache, and returns the Astropy table.

read_cached_or_fetch_table()

Fetches or loads one catalogue table.

keyed_table()

Converts an Astropy table into a dictionary keyed by a selected name column.

keyed_herrera_bars()

Filters Herrera-Endoqui rows to keep only Type == bar, then keys those rows by Name.

fetch_catalog_geometry()

Fetches all three public catalogues and returns dictionaries for Herrera bars, Salo galaxies, and Diaz-Garcia Fourier data.

FITS Metadata

fits_metadata() checks whether the expected FITS file exists. If it does, it reads the FITS header and extracts image dimensions, pixel scale, reference pixel, sky coordinate reference values, and image position angle. If the image is missing, the manifest still gets a row, but image_exists is False.

```
image_naxis1 / image_naxis2
pixel_scale_arcsec_x / pixel_scale_arcsec_y
crpix1 / crpix2
crval1_deg / crval2_deg
image_pa_axis2_deg
```

Manifest Builder

`build_manifest()` is the core function. It loops over every galaxy in the scrambled map, constructs the expected image filename, looks up matching rows in all available data sources, records notes for missing matches, then fills a row matching `OUTPUT_FIELDS`.

- Salo provides center coordinates, disk position angle, disk ellipticity, and disk radial bounds.
- Herrera-Endoqui provides bar position angle, bar semi-major axis, and bar ellipticity.
- The local S4G table provides inclination, mass, distance, radii, rotation velocity, and several bar-size variants.
- Diaz-Garcia values are preferred for A2, A4, and deprojected bar ellipticity when present; local S4G values are used as fallback.

```
"bar_A2": diaz.get("A2") or s4g.get("A2")  
"bar_A4": diaz.get("A4") or s4g.get("A4")
```

Writing and CLI

`write_manifest()`

Creates the output directory if needed, then writes rows with `csv.DictWriter` using `OUTPUT_FIELDS` as the column order.

`parse_args()`

Defines command-line options for overriding input paths, output path, cache directory, Vizier behavior, and cache refresh behavior.

`main()`

Parses arguments, optionally fetches Vizier data, builds the manifest, writes the CSV, then prints row and image-link counts.

```
python build_s4g_geometry_manifest.py  
python build_s4g_geometry_manifest.py --no-vizier  
python build_s4g_geometry_manifest.py --refresh-cache
```

Small Oddity

`S4G_COLUMNS_TO_KEEP` is defined but not used by the current implementation. It looks like a leftover or planned filter list. The current code reads all local table columns and only emits selected values through `OUTPUT_FIELDS`.

Summary

In short, the module is a data-joining pipeline: local galaxy list plus local S4G measurements plus public catalogue geometry plus FITS header metadata become one clean CSV manifest, one row per galaxy.